Fine-tuning et RAG

EMSI - Université Côte d'Azur
Richard Grin
Version 2.12 – 8/11/26

1

Plan du support

- Fine-tuning
- RAG
- Comparaison prompt engineering, fine-tuning, RAG
- Bases de données vectorielles
- RAG avec LangChain4j
- Références

2

Principaux problèmes LMs publics

- Ils peuvent halluciner
- Pas de connaissances sur les données privées des entreprises car ils n'y ont pas eu accès pendant leur apprentissage
- Leurs connaissances s'arrêtent à la date de fin de leur apprentissage
- Ils ne peuvent donc pas être utilisés tels quels dans une application d'entreprise

3

Pour atténuer les problèmes

- Prompt engineering (pour atténuer les hallucinations et espérer de meilleures réponses)
- Fine-tuning qui modifie le LM (change ses paramètres) par un entraînement complémentaire sur des données non vues lors du pré-entraînement
- RAG qui ne modifie pas le LM mais qui, pour chaque question,
  - recherche informations supplémentaires dans des documents fiables
  - les ajoute au prompt pour que le LM puisse les utiliser

4

Fine-tuning

5

Fine-tuning (réglage fin)

- **Entraînement complémentaire** d'un modèle d'IA existant (OpenAi, Gemini)
  - Le pré-entraînement du modèle a permis d'apprendre les structures de langage, la syntaxe, la sémantique et des connaissances générales
  - Le fine-tuning **modifie les paramètres du modèle** ; le nouveau modèle acquiert des connaissances sur une tâche spécifique, dans un domaine particulier, ou apprend à converser avec un style ou un ton défini

6

Entraînement pour fine-tuning

- On fournit à l'API des **paires de texte** (entrée, et sortie attendue) pour entraîner le modèle
- Chaque LM a son propre format pour les données d'entraînement
- Pour avoir un impact significatif sur le modèle de base pré-entraîné, il faut l'entraîner sur au moins plusieurs centaines (pour une tâche très spécifique) à plusieurs milliers d'exemples

Un exemple (format Gemini)

```
{
  "contents": [
    {
      "role": "user", # Donnée en entrée
      "parts": [
        {
          "text": "..."
        }
      ]
    },
    {
      "role": "model", # Réponse attendue du modèle
      "parts": [
        {
          "text": "..."
        }
      ]
    }
  ] #  ... répéter des exemples avec le couple user - model.
}
```


Coûts

- L'entraînement du fine-tuning est bien plus coûteux que l'utilisation du modèle (inférence) ; il nécessite des machines puissantes et beaucoup de temps
- Les données utilisées pendant l'entrainement sont longues et complexes à créer, avec intervention humaine
- Le fine-tuning est de loin la solution la plus coûteuse pour modifier le comportement du modèle



Options pour le fine-tuning

- Le fine-tuning étant long et très coûteux, des options ont vu le jour (soft-prompting, T-few, LORA, adapters, pré-entraînement continu, ...)
- Ces options offrent un compromis intéressant entre coût et personnalisation mais n'égalent pas la performance d'un fine-tuning complet sur des tâches complexes

Risque du fine-tuning

- Si les exemples du fine-tuning ne concernent que le domaine pour lequel on veut rendre le modèle meilleur, il y a un risque que le modèle soit moins bon pour les autres domaines
- Il est conseillé de ne pas trop forcer vers le nouveau domaine ; par exemple, ne pas choisir que des exemples qui concernent ce nouveau domaine



Retrieval-Augmented Generation (RAG)

- Bases
- RAG classique
- Techniques avancées pour RAG classique
- Autres types de RAG

Bases

13

Présentation

- Le RAG est une technique qui permet de fournir à un LM des données externes fiables pour lui permettre de répondre aux questions
- Ces données peuvent être contenues dans des documents d'entreprise ou des documents récents, sur lesquels le LM n'a pas été entraîné
- Le LM peut alors répondre à des questions en se basant sur ces documents et, au besoin, en indiquant les sources d'information qu'il a utilisées pour répondre

14

Utilisation des données

- Au contraire du fine-tuning, les données fournies au LM ne modifient pas les paramètres du LM
- Pour chaque question posée, des informations liées à la question sont automatiquement ajoutées au contexte pour aider le LM à répondre

15

Sources des données externes

- Fichiers de tous types (Markdown, PDF, …)
- Articles
- Podcasts
- Vidéos
- Recherches sur Internet
- BD structurées (relationnelles) ou semi-structurées (noSQL)
- Graphe de connaissances géré par une BD graphe
- API ou moteurs de recherche (API d'encyclopédies, Google, Bing)
- …

16

2 grandes phases pour le RAG

- Ces 2 phases sont **indépendantes** et se déroulent le plus souvent à des moments différents :
  1. Enregistrement des documents
  2. Fournir certains de ces documents au LM pour l'aider à répondre à une question

17

Phase 1 : ingestion documents

- Les documents utiles pour aider le LM à répondre aux questions sont ajoutés dans le système de RAG, le plus souvent dans une base de données
- Il est important de préparer les documents afin d'améliorer l'efficacité de la phase suivante
- Cette préparation peut comporter
  - nettoyage des documents
  - découpage en segments des documents
  - ajout de métadonnées



18

Préparation des données (1/2)

- Pendant la phase d'ingestion, le choix et la préparation des documents est très importante car elle permet d'améliorer grandement les résultats et d'économiser des ressources
- Le RAG participe à la réduction des hallucinations en considérant fiables les documents retrouvés ; il faut donc commencer par s'assurer de la qualité des informations contenues dans les documents
- Le processus peut être complexe mais il ne s'effectue qu'une seule fois, quand les documents sont ajoutés

19

Préparation des données (2/2)

- Etapes du processus :
  - Vérifier la justesse et le bon niveau de détails des informations contenues dans les documents.
  - Nettoyer les documents pour les uniformiser et enlever les parties qui ne seront pas utiles
  - Découper les documents en morceaux/segments (chunks)
  - Des métadonnées peuvent être ajoutées pendant le processus ; par exemple ajouter aux segments le titre du document global, et leur index dans le découpage

20

Nettoyage des documents

- Supprimer éléments non pertinents (pour les types de questions qui seront posées) : en-têtes, pieds de page, signatures automatiques des emails, mentions légales, informations répétées à chaque page (nom de l'auteur, titre du document, du chapitre, date), publicités, éléments de mise en forme (par exemple dans pages HTML), liens externes, …
- Corriger fautes d'orthographe et de grammaire
- Les données spéciales (tables, images) peuvent nécessiter des traitements supplémentaires
- Réécrire les passages importants ambiguës

21

LM dans la préparation

- Un LM peut être très utile pendant le nettoyage des documents
- Un LM peut, par exemple, corriger les fautes d'orthographes, repérer les éléments de mise en forme non essentiels, faire des transformations sur les tables

22

Chunk (morceau/segment) (1/2)

- Les documents externes sont presque toujours découpés en segments pendant la phase d'ingestion
- On peut ainsi mieux cerner les passages des documents qui sont pertinents pour la question posée
- On réduit aussi le volume de données à traiter, ce qui donne des réponses meilleures, plus rapides et une meilleure gestion des ressources et des coûts
- Les segments sont le plus souvent de taille fixe (300 à 800 tokens est une taille courante) ; des tests sont souvent nécessaires pour trouver la meilleure taille

Pourquoi ?

23

Chunk (morceau/segment) (2/2)

- Les segments peuvent se chevaucher légèrement (par exemple 10 à 15 % de la taille des chunks) pour éviter une perte de contexte
- Tous les segments peuvent ne pas avoir la même taille ; essayer de tenir compte des phrases et des paragraphes ; si le document est structuré, essayer de tenir compte des structures (sections par exemple) ; un LLM peut aider à découper en segments cohérents (plus coûteux et plus long)

24

Phase 2 : génération réponse

- Décomposée en 2 phases consécutives quand une question est posée à l'IA :
  1. Récupération (retrieval) et augmentation : parmi les données ingérées dans la phase 1, le système retrouve celles qui sont les plus pertinentes pour répondre à la question, et les ajoute à la question
  2. Génération : le tout (données pertinentes + question + éventuellement historique de la conversation) est envoyé au LM qui génère une réponse

Paramètres à définir dans le RAG

- Taille des segments (ou manière de les définir)
- Nombre de segments ajoutés à la question
  - Un trop grand nombre peut nuire à la qualité car les informations importantes risquent d'être noyées dans des informations moins intéressantes
  - Un trop petit nombre risque de manquer des informations importantes
- Il faut tester pour choisir les valeurs des paramètres qui donnent les meilleurs résultats

Avantages du RAG

- Réduction des hallucinations : le LM peut s'appuyer sur des sources fiables et adaptées aux types de questions prévus pour l'application
- Accès à des informations privées de l'entreprise
- Accès à des informations à jour, plus récentes que la fin de l'entraînement du LM
- Plus souple et moins coûteux que le fine-tuning
- Meilleur explicabilité des réponses du LM car les sources utilisées peuvent être citées

Exemples d'utilisation du RAG

- Service client intelligent ; chatbot pour répondre aux clients de l'entreprise
- Interroger le LM sur des fichiers PDF qui contiennent les règles de fonctionnement d'une entreprise
- Interroger le modèle sur le contenu d'une vidéo
- Assistance juridique en s'appuyant sur des lois, des règlements et la jurisprudence
- Aide à l'apprentissage ou à la recherche dans un domaine particulier
- Support technique avancé avec manuels techniques

Température du LM

- Pour le RAG il est conseillé de réduire la température du LM pour l'inciter à ne pas trop faire preuve de « créativité », à ne pas « divaguer »
- En effet, un des buts principaux du RAG est d'obtenir des résultats fiables, de limiter les hallucinations

Priorité informations récupérées

- Un LM donne la priorité aux informations récupérées par le RAG, par rapport aux connaissances apprises

  Comment c'est possible ?
- L'agent IA qui fait le RAG ajoute les données retrouvées par le RAG dans le prompt en indiquant que ces données sont prioritaires par rapport aux données apprises

RAG dynamique

- Les documents ajoutés au contexte du LM peuvent ne pas provenir d'une base statique ; ils peuvent être récupérés avec une ou plusieurs APIs ou du Web, par exemple

31

Exemple RAG dynamique

Quel est le chiffre d'affaires du client XXX cette année ?
L'agent IA :

1. interroge le CRM
2. récupère les ventes
3. les convertit en texte ou en tableau
4. les ajoute au contexte du LLM



32

Comparaison RAG dynamique - outils

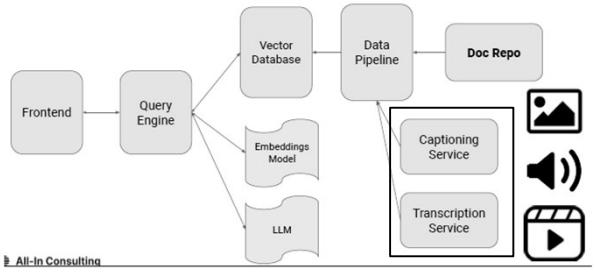
- Les données dynamiques peuvent aussi être retrouvées avec des outils
- Différence essentielle :
  - Avec RAG dynamique, tout est orchestré par l'agent IA
  - Avec les outils, c'est le LM qui décide quels outils utiliser
- Lorsque l'on sait exactement où trouver l'information que l'on veut, le RAG dynamique est préférable car il est plus sécurisé (déterministe) et moins coûteux (en tokens et réflexion du LM)
- Sinon, l'utilisation d'outils est plus souple

33

RAG multi-modal

- Prend en compte non seulement du texte mais aussi d'autres types de media comme l'audio, la vidéo, les images
- A l'architecture de base il faut ajouter des traitements pour prendre en compte ces médias car les LMs sont centrés essentiellement sur le texte (et un peu les images)
- Par exemple, les vidéos peuvent être représentées par des captures d'écran et par la transcription de la bande audio

34



35

RAG récursif

- Une requête peut être obtenue plus efficacement par plusieurs résultats intermédiaires qui sont agrégés pour obtenir une réponse
- Les étapes intermédiaires permettent de mieux cerner les besoins et d'aider le LM à utiliser les bonnes informations ; les 1ères étapes peuvent aussi influencer les étapes suivantes pour optimiser la recherche

36

Exemple

- Une 1ère étape recherche dans une base qui contient des résumés d'articles, ce qui indique dans quels articles rechercher les détails de l'information cherchée
- Voir LlamaIndex (https://www.llamaindex.ai/)

- Dans la suite on appellera
  - « document » une source d'information externe, qui pourra être un document entier ou, le plus souvent, un « chunk », un segment de document, obtenu par découpage en morceaux d'un document
  - « question » une question posée à un LM avec, éventuellement, l'historique de la conversation

RAG « classique »

Divers types de RAG

- Il existe de nombreuses formes de RAG
- La suite de cette section présente le RAG « classique » ou « sémantique » qui utilise des embeddings pour enrichir le contexte fourni au LM
- Ce RAG intervient dans presque toutes les formes de RAG, combiné à des techniques avancées ou à d'autres types de RAG (étudiés plus loin)

RAG classique

- Utilise les **embeddings** pour retrouver les informations pertinentes pour répondre à une question
- Les documents sont enregistrés dans une base de données vectorielles, avec leur embedding
- Quand une question est posée
  1. L'embedding de la question est calculé
  2. Les documents qui ont un embedding similaire sont récupérés dans la BD vectorielle
  3. La question et ces documents sont envoyés au LM

Phases pour un RAG classique

Phase 1 :



Explications ?

Phase 2 :

Question +



Explications ?

(Source Oracle University)

Enregistrement des documents

- L'embedding de chaque document (ou segment de document) est calculé par un modèle d'embedding
- Les couples embedding-document sont enregistrés dans un magasin d'embeddings, le plus souvent une base de données vectorielle
- Une BD vectorielle est optimisée pour faire des recherches de similarité très rapides

43

Quand a lieu cette phase 2 ?

Phase 2 : sélection des données

- Il n'est pas envisageable d'ajouter dans chaque prompt envoyé au LM toutes les données ingérées Pourquoi ?
- Dans la phase 2, il faut donc un moyen de retrouver les données les plus pertinentes pour répondre à la question
- Le RAG classique retrouve les documents qui ont un embedding proche de l'embedding de la question, c'est-à-dire les documents les plus similaires à la question

44

Etapes récupération - génération

- La question est transformée en embedding avec le même modèle d'embedding que les documents enregistrés dans l'étape d'ingestion
- Les n (par exemple 10) documents qui ont les embeddings les plus similaires à l'embedding de la question sont ajoutés au prompt, le plus souvent devant la question, les plus pertinents en premier
- Une étape optionnelle de « reranking » peut réordonner ces n documents plus finement dans le prompt
- Le tout est envoyé au LM qui répond à la question

45

Evaluation des performances

- Il est important de pouvoir juger de la qualité des résultats pour choisir les meilleures valeurs des paramètres en testant sur des questions dont on connait les réponses
- Un autre LM peut aussi aider à juger de la qualité des réponses fournies par le RAG

46

Avantage recherche sémantique

- Elle permet de capturer les sens des mots et des textes, ce qui est souvent bien plus souple que la correspondance exacte avec des mots-clés
- Cet avantage est essentiel
- Malgré tout, la recherche sémantique a des inconvénients qui font qu'elle est souvent combinée à d'autres techniques

47

Inconvénients recherche sémantique

- Quelquefois moins efficace que recherche par mots exacts ; par exemple « Pourquoi code erreur E3345 ? »
- Similarités parfois trompeuses ; « Comment installer PostgreSQL ? » peut retrouver « Installation MySQL »
- Raisonnements complexes pas pris en compte ; « Quels produits vendus après 2025 ont un prix supérieur à ceux vendus en 2024 ? »
- Pas une vision globale d'un document à cause du découpage en segments

48

Techniques avancées pour RAG classique

Décomposition de la question

- Certaines questions peuvent être décomposées pour lancer plusieurs branches de récupération en parallèle
- La décomposition en branches peut être effectuée par un LM, par des règles métier qui détectent des mots-clés (quand les questions concernent un domaine restreint et bien connu) qui correspondent à une classification des documents utilisés par le RAG, en parcourant un graphe de connaissance, …
- Les résultats des branches sont ensuite fusionnés

Utilisation de l'historique

- Le plus souvent, la recherche des morceaux les plus pertinents n'utilise que la dernière question posée, et pas tout l'historique de la conversation
- Il peut être intéressant de modifier la question posée, par exemple pour tenir compte de l'historique de la conversation (voir QueryTransformer)

Reranking (reclassement)

- Solution possible si une recherche sémantique ne donne pas un résultat satisfaisant
- Un reclassement peut permettre d'obtenir un meilleur résultat : les documents retrouvés sont réexaminés plus finement pour avoir un résultat plus pertinent ; on peut réordonner les documents mais surtout ne garder que les meilleurs
- On peut ainsi appliquer une méthode plus précise, mais plus lourde et plus coûteuse, sur un nombre limité d'items

Routage

- Souvent les données privées des entreprises sont conservées dans des endroits et des formes diverses
- Plutôt que de parcourir toutes les sources de données à chaque prompt, un routage permet de ne consulter que certaines de ces sources de données
- Le routage peut s'appuyer sur
  - Des règles diverses (autorisations de l'utilisateur, service qui a émis la requête, …)
  - Des mots-clés
  - Des calculs de similarité
  - Un choix fait par un LM

Pas de RAG inutile

- Une question peut ne pas nécessiter de RAG
- Par exemple, « Quelle est la capitale de la France ? »
- Lancer un RAG serait inutilement coûteux
- Peut être considéré comme un type particulier de routage qui peut s'appuyer sur les mêmes aides à la décision (RAG ou pas RAG ?)

Métadonnées

- Peuvent être enregistrées avec les segments de texte et les embeddings pour accélérer la récupération, améliorer la pertinence des documents et la qualité de la réponse
- Peuvent contenir le titre du document, sa source (livre, site Web, …), la date de publication, les auteurs, …
- Au moment de la récupération, elles offrent la possibilité de filtrer des documents avant la recherche par mots-clés ou par similarité
- Les BD vectorielles permettent souvent d'associer des métadonnées aux embeddings

55

Exemples utilisation métadonnées

- Id des documents ; permet de citer les documents d'où sont extraits les informations retrouvées par le RAG
- Catégorie : Filtrer par sous-domaine du droit (par exemple, droit civil, droit commercial, droit du travail)
- Date : Filtrer par date de publication d'articles scientifiques pour ne garder que les articles récents ou ceux publiés dans une certaine période
- Confidentialité : Filtrer les documents selon leur niveau de confidentialité pour restreindre l'accès (public, interne, confidentiel)



56

Utilisation d'outils

- Des outils, par exemple pour effectuer des calculs complexes ou pour retrouver les cours de devises, peuvent être utilisés pendant la phase de génération de la question

57

Autres techniques (1/2)

- Réécriture de la question par un LM : il peut, par exemple, réécrire la question avec des termes plus adaptés aux documents enregistrés, utiliser l'historique pour des ambiguïtés (déjà vu) ou utiliser un vocabulaire plus « métier »
- Hyde (Hypothetical Document Embeddings) : un LM écrit une réponse hypothétique (pas nécessairement correcte) et le RAG recherche les embeddings similaires à cette réponse hypothétique

58

Autres techniques (2/2)

- CRAG (Corrective RAG) : un évaluateur de documents retrouvés intervient avant l'envoi au LM de la question et du contexte ; si le score est faible, l'agent intervient (réécriture de la question, recherche Web, …) pour essayer de récupérer de meilleurs documents
- Self-RAG va un peu plus loin : le LM est entrainé pour s'auto-évaluer, pour décider si les documents retrouvés sont suffisants ; plus complexe et plus coûteux (entraînement nécessaire) donc plus rarement utilisé

59

Granularité recherche RAG

- RAG sequence model : technique la plus utilisée car la plus simple et suffisante dans la plupart des cas ; les documents les plus pertinents sont récupérés et ajoutés à la question et à l'historique de la conversation ; le tout est envoyé au LM qui génère les tokens de la réponse
- RAG token model : souvent plus précis mais plus difficile à mettre en œuvre et plus coûteux; chaque token est généré par le LM en tenant compte des documents les plus pertinents pour les tokens de la réponse déjà générés ; intéressant quand différentes parties de la réponse sont associées à des documents différents

60

Autres types de RAG

61

Divers types de RAG (1/2)

- RAG « classique » ou « sémantique » s'appuie sur les embeddings et les BD vectorielles
- D'autres types de RAG diffèrent par la manière d'enrichir le contexte avec des informations liées à la question posée
- Le RAG hybride est très commun ; il combine une recherche sémantique avec une recherche par mots exacts (par mots-clés)
- Graph RAG s'appuie sur un graphe de connaissances

62

Divers types de RAG (2/2)

- « LLM Wiki pattern » s'appuie sur un Wiki de connaissances enrichi progressivement
- Souvent, les autres types de RAG sont combinés avec du RAG classique pour récupérer les documents détaillés (emails, contrats, fiches process, …)

63

Recherche par mots exacts

- Recherche de certains mots dans les documents
- Peut éviter les approximations de la recherche sémantique (confusion entre concepts proches)
- Les BD vectorielles les plus utilisées permettent de combiner une recherche sémantique et par mots exacts (recherche lexicale)
- Les documents/segments enregistrés pour le RAG sont prétraités pour l'utilisation d'un index inversé qui indique pour chaque mot-clé les documents qui le contiennent, avec un score « BM25 » qui indique à quel point chaque document est important pour ce mot-clé

64

TF-IDF, BM25

- BM25 (Best Match 25), amélioration de TF-IDF (*Term Frequency-Inverse Document Frequency*), permet une recherche pas mots exacts
- TF-IDF : les mots d'un document ont un poids fonction
  - de leur fréquence dans le document (TF)
  - de leur fréquence inverse dans l'ensemble des documents (IDF)
- Les mots qui apparaissent fréquemment dans un document, mais rarement dans d'autres, sont considérés comme particulièrement significatifs

65

Graphe de connaissances (1/2)

- Type de base de connaissances dans lequel les connaissances sont structurées sous la forme de graphe
- Contient des entités et des relations entre ces nœuds
- Exemples :
  - [Paris] (entité de type ville) est la capitale de (relation) [la France] (entité de type pays)
  - [Marie Curie] a gagné le [prix Nobel de physique] en [1903]
- Exemples de graphe de connaissances : Google Knowledge Graph (peut être affiché à droite des résultats d'une recherche Google), Wikidata

66

Graphe de connaissances (2/2)

- Permet de faire des déductions et d'obtenir des informations liées à une entité en parcourant le graphe
- Très flexible : facile d'ajouter des nouveaux types de nœuds et de liens
- Le contenu d'un graphe de connaissances peut être extrait automatiquement de textes non structurés avec la NER (Named Entity Recognition) pour trouver les entités et la RE (Relation Extraction) pour trouver les liens ; un LM (entraîné pour cela) peut être utilisé

RAG avec graphe de connaissances

- Appelé « Graph RAG »
- Les informations qui pourraient être utiles au LM pour répondre sont extraites d'une base Graphe au lieu d'une base vectorielle
- Le reste est identique au RAG classique : les informations récupérées sont ajoutées à la question et le tout est envoyé au LM
- Variantes, suivant qu'un LM intervient ou non dans la recherche d'informations

Variantes de Graph RAG

- Dépend du degré d'intervention d'un LM
- Dans certains cas particuliers (domaine restreint), une requête prédéfinie paramétrable peut rechercher les informations
- Un logiciel NER extrait les entités et relations de la question et ensuite le graphe de connaissances est parcouru avec une certaine profondeur
- Un LM est utilisé pour générer les requêtes qui extraient les informations

Combiner recherche vectorielle et recherche dans graphe connaissances

- Un graphe de connaissances est particulièrement efficace pour répondre à des questions qui nécessitent de suivre des liens entre plusieurs concepts
- Cependant, il ne contient généralement qu'une représentation synthétique des informations ; à l'inverse une recherche vectorielle conserve l'information textuelle détaillée
- Les systèmes Graph RAG combinent souvent les 2 recherches (effectuées en //) pour fournir le contexte enrichi au LM

Wiki de connaissances

- Représentation structurée et évolutive des connaissances d'un domaine, organisée en pages reliées entre elles, que des humains ou une IA peuvent créer, enrichir et consulter
- Chaque page peut représenter un concept, une procédure, un objet ; contient des liens vers d'autres pages associées
- Support de persistance pas imposé ; peut être des fichiers Markdown, une BD relationnelle ou graphe, …



Prompt engineering vs fine-tuning vs RAG

- Comparaison entre les 3 façons de prendre en compte des données sur lesquelles un LM n'a pas été entrainé

73

Avantages et inconvénients (1/3)

- **Prompt engineering** :
  - simple, souple et rapide
  - **mais**
    - difficile de mettre à l'échelle (solution ad hoc pas généralisable ; peut dépendre des versions des LMs)
    - résultats aléatoires
    - le LM doit déjà connaitre le domaine concerné par la question

74

Avantages et inconvénients (2/3)

- **Fine tuning** :
  - performant
  - pas limité par la taille maximale du contexte
  - parfait pour répondre avec un certain style
  - expert dans un domaine
  - **mais**
    - demande une préparation lourde, coûteuse en temps et argent
    - cette préparation est à répéter quand les données changent
    - moins précis et moins souple que le RAG
    - danger de perte de capacités générales

75

Avantages et inconvénients (3/3)

- **RAG** (souvent la meilleure solution) :
  - permet de réduire les hallucinations en fournissant des informations fiables avec des sources vérifiables
  - peut fournir des informations différentes selon le type d'utilisateur, par exemple en réservant des informations sensibles à des utilisateurs privilégiés
  - permet de tester facilement plusieurs LMs
  - **mais**
    - traitement à répéter à chaque recherche (donc moins rapide que le fine tuning)
    - parfois difficile à mettre en œuvre
    - pas adapté pour certaines tâches, par exemple parler en imitant une personne ou répondre avec un certain style
    - lourd à gérer si documents très nombreux

76

Que choisir ? (1/4)

- Le prompt engineering seul ne peut convenir que si le LM connait déjà le domaine de la question

77

Que choisir ? (2/4)

- Le fine tuning convient bien pour
  - répondre à des questions dans un domaine bien défini et spécifique avec des informations **stables dans le temps** ; par exemple assistance technique sur des produits de base, compréhension de termes médicaux
  - générer du contenu qui n'est pas basé sur des informations incluses dans des documents ; par exemple, imiter une voix, adopter un certain ton ou style
  - reconnaitre des patterns ou des sentiments ; par exemple, classer des emails envoyés par des clients : problèmes techniques, retours, satisfaction, …

78

Que choisir ? (3/4)

- Le RAG convient bien pour
  - des tâches nécessitant des informations actualisées en temps réel ; par exemple, assistant pour aider les clients à investir en bourse
  - obtenir des réponses basées sur des documents d'entreprise qui peuvent être modifiés à tout moment ; par exemple, FAQ, manuels de procédures
  - outil de recherche dans des bases très volumineuses qui sont continuellement mises à jour, recherche sur le Web ; par exemple recherche d'articles scientifiques à jour

En résumé, que choisir ? (4/4)

- Le RAG permet d'**améliorer les réponses d'un modèle** grâce à des informations supplémentaires pertinentes et à jour
- Mais il **ne modifie pas fondamentalement le comportement ou le style linguistique du modèle ;** en ce cas, le fine-tuning sera meilleur
- Le Fine-tuning ne permet pas d'actualiser rapidement le modèle avec de nouvelles informations et il est coûteux en temps et en argent
- Le plus souvent, le RAG, ou une combinaison des 2 techniques, sera donc la meilleure solution

Combinaison des techniques

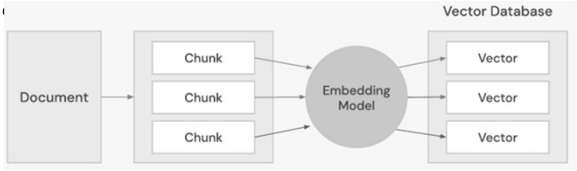
- Le prompt engineering, le fine-tuning et le RAG peuvent être combinés sur un même projet pour en tirer le meilleur parti ; par exemple

1. Commencer par le plus simple, avec des techniques de prompt engineering (utiliser des templates)
2. Ajouter du RAG s'il manque des informations pour bien répondre
3. Faire du fine-tuning sur le LM utilisé par le RAG pour le spécialiser dans un domaine ou/et si le ton, le style ou le format des réponses ne conviennent pas

Bases de données vectorielles

BDs vectorielles

- Fréquemment utilisées pour le RAG
- BD pour stocker et rechercher des vecteurs de données
- Ces BDs peuvent faire des recherches par similarités, plutôt que de trouver des correspondances exactes



Principales caractéristiques

- Stockage et gestion d'un très grand nombre de vecteurs (jusqu'à des milliards)
- Recherche rapide des vecteurs similaires à un vecteur (recherche ANN, Approximate Nearest Neighbor) ; indexation adaptée à cette recherche
- Stockage et gestion de métadonnées associées aux vecteurs ; par exemple titre, date de parution, auteurs, emplacement du contenu d'articles
- Support multimodal (texte, image, audio)

Index

- Les vecteurs sont indexés pour permettre des recherches de similarité rapides
- Utilisent des techniques spéciales pour accélérer les recherches et réduire l'occupation de la mémoire

85

Exemples d'utilisation

- Exemple de base : recherche des 10 embeddings les plus proches d'un embedding (top_k = 10)
- Recherche sémantique (pour RAG en particulier)
- Recommandation de contenu (pour achats de produits similaires)
- Recherche d'images ou de vidéos (par caractéristiques visuelles)

86

Contenu de la BD vectorielle

- Une BD vectorielle peut stocker les textes associés aux embeddings ou ne stocker que les informations nécessaires pour retrouver ces textes ailleurs (par exemple, dans une base de données classique ou un système de stockage)
- Elle peut aussi contenir des métadonnées qui peuvent alors servir à filtrer les résultats des requêtes (date de création, mots-clés, catégories,…)

87

Types de recherche

- Les recherches dans les BD vectorielles sont le plus souvent des recherches sémantiques, appelées aussi recherches denses, qui utilisent des embeddings
- De nombreuses BD vectorielles peuvent aussi faire des recherches par mots-clés avec des filtres basés sur les métadonnées
- Ces BD vectorielles peuvent faire des recherches hybrides qui combinent la précision et la rapidité des recherches par mots-clés et la compréhension du sens des recherches sémantiques

88

Exemples de recherche hybride

- Si les données sont très nombreuses, un système peut effectuer d'abord une recherche rapide par mots-clés pour réduire le nombre d'items et ensuite faire une recherche sémantique sur le résultat
- Utiliser une recherche par mots-clés pour obtenir un premier ensemble de résultats, puis réordonner ces résultats en fonction des similarités de vecteurs des documents
- Il est possible d'attribuer un poids à chacune des 2 recherches pour obtenir le résultat final
- Les résultats d'une recherche dense peuvent être filtrés pour éliminer les items qui n'ont pas certains mots-clés

89



BD vectorielle en mémoire

- Les TPs utilisent une BD en mémoire très simple `InMemoryEmbeddingStore` qui évite l'installation d'une « vraie » BD vectorielle
- Grâce à LangChain4j, il est simple d'utiliser à la place une « vraie » BD vectorielle car le code Java est identique, à part la création de `l'EmbeddingStore`
- Dans une application d'entreprise il faut utiliser une « vraie » BD vectorielle (LangChain4j en supporte de nombreuses) pour ne pas perdre les embeddings entre 2 sessions de travail et profiter des index

90

Des BDs supportées par LangChain4j

- Chroma
- Milvus
- Weaviate
- Pinecone
- Vespa (Yahoo!)
- Qdrant
- Elasticsearch (avec plugins spécialisés)
- AI Vector Search (Oracle)
- Redis
- Astra DB
- Cassandra



Chroma

- BD vectorielle gratuite (https://www.trychroma.com/home) et open source (https://github.com/chroma-core/chroma)
- On peut l'installer en local ; le plus simple est d'utiliser une image Docker
- Fournit une API utilisable à distance mais le plus simple est de l'utiliser par l'intermédiaire de LangChain4j

Chroma - Installation

- Le plus simple est de l'installer avec Docker en tapant cette commande dans un terminal : docker pull chromadb/chroma
- Une autre façon est d'aller dans Docker Desktop et de cliquer sur l'icône de Docker Hub pour chercher « chromadb » pour récupérer chromadb/chroma
- Dans Docker Desktop on peut alors lancer l'exécution de chromadb/chroma en cliquant sur le triangle « Run » ; dans la fenêtre qui s'ouvre, clic sur « Optional settings » et associer alors les ports 8000 de l'hôte et du container

Chroma - Utilisation (1/2)

- Dans pom.xml :

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-chroma</artifactId>
    <version>****</version>
</dependency>
```


Chroma - Utilisation (2/2)

- Pour créer l'`EmbeddingStore` :

```
EmbeddingStore<TextSegment> embeddingStore =
    ChromaEmbeddingStore.builder()
    .baseUrl("http://localhost:8000/")
    .collectionName("ma-collection")
    .build();
```

- Tout le reste est identique à l'utilisation de `InMemoryEmbeddingStore`

Code pour utiliser Chroma

```
EmbeddingStore<TextSegment> embeddingStore =
    ChromaEmbeddingStore.builder()
        .baseUrl("http://localhost:8000")
        .collectionName("ma-collection")
        .build();
  // Tout le reste du code ne dépend pas de Chroma
  EmbeddingModel embeddingModel = ...;
  ...
}
```

RAG avec LangChain4j

- Généralités et RAG « facile »
- Classes et interfaces LangChain4j

97

Présentation

- LangChain4j fournit plusieurs types Java pour faciliter le RAG (https://docs.langchain4j.dev/tutorials/rag)
- On verra d'abord un « RAG facile » pour faire du RAG sans configurations ni optimisations
- Les classes qui permettent de construire des RAGs plus adaptés aux cas particuliers seront étudiées ensuite

98

Phase 1 du RAG - Ingestion

- On indique
  - L'origine des données externes utilisées pour le RAG (base de données, documents d'entreprise,...)
  - Le magasin d'embeddings où seront entreposés les morceaux et leurs embeddings



99

Exemple RAG facile (1/3)

```
List<Document> documents = FileSystemDocumentLoader.
    loadDocuments("/home/langchain4j/documentation");
EmbeddingStore<TextSegment> embeddingStore =
    new InMemoryEmbeddingStore<>();
EmbeddingStoreIngestor.ingest(documents, embeddingStore);
```

Pour que ce code fonctionne, il faut ajouter une dépendance pour langchain4j-easy-rag

100

Types du code

- `Document` : un texte provenant d'un fichier, avec éventuellement des métadonnées ; par exemple le contenu d'un fichier local ou d'une page Web
- `FileSystemDocumentLoader` : charge des fichiers pour obtenir des `Document`
- `TextSegment` : segment de texte (chunks) et métadonnées associées Vous devinez lesquelles ?
- `EmbeddingStore<TextSegment>` : magasin d'embeddings de `TextSegment`, avec les segments et d'éventuelles métadonnées
- `EmbeddingStoreIngestor` : Pipeline pour l'ingestion de documents dans un magasin d'embeddings

101

Phase 2 du RAG



102

Exemple RAG facile (2/3)

- Interface Java qui représente l'interface avec le LM ; par exemple

```
public interface Assistant {
  String chat(String userMessage);
}
```

Interface implémentée en un assistant IA par la classe `AiServices` fournie par LangChain4j

Exemple RAG facile (3/3)

```
ChatModel model = …;
Assistant assistant = AiServices.builder(Assistant.class)
    .chatModel(model)
    .chatMemory(MessageWindowChatMemory.withMaxMessages(10))
    .contentRetriever(
        EmbeddingStoreContentRetriever.from(embeddingStore))
    .build();

String reponse =
  assistant.chat("Comment annuler une commande ?");
```

Indique le modèle de chat et le magasin d'embeddings utilisé pour faire du RAG, ce qui permet à `AiServices` d'implémenter correctement le code de l'assistant

Types du code

- `EmbeddingStoreContentRetriever` : retrouve dans un magasin d'embeddings les `Content` (enveloppes de `TextSegment`) les plus similaires à la question



Afficher le contenu des sources

- Pour afficher le contenu des sources utilisées par le RAG, utiliser un assistant dont la méthode retourne `Result<String>` (au lieu de `String`) et appeler la méthode `Result.sources()`

Exemple

```
Result<String> reponse = assistant.chat(question);
System.out.println("Assistant : " + reponse.content());
System.out.println("Sources : ");
for (Content source : reponse.sources()) {
  System.out.println(source.textSegment());
}
```

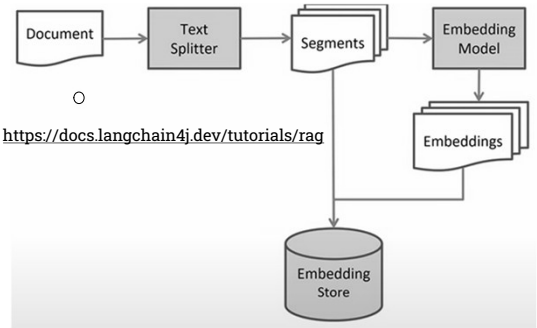



Classes et interfaces pour RAG

- Phase 1 : chargement (ingestion) des `Documents`
- Phase 2 : récupération (retrieval) des données pertinentes et augmentation du prompt
- RAG avancé

RAG - Ingestion

Explications ?



Source https://docs.langchain4j.dev/tutorials/rag

109

Exemple schématique simple

```
// Création d'un document avec un DocumentParser
DocumentParser documentParser = ... ;
Document document = UnDocumentLoader.loadDocument(
    cheminFichier, documentParser);
// Découper le document en segments (chunks)
DocumentSplitter splitter =
    DocumentSplitters.recursive(300, 20);
List<TextSegment> segments = splitter.split(document);
// Convertit les chunks/segments en embeddings
EmbeddingModel embeddingModel = ... ;
List<Embedding> embeddings =
    embeddingModel.embedAll(segments).content();
// Ajoute embeddings et chunks dans magasin embeddings
EmbeddingStore<TextSegment> embeddingStore = ... ;
embeddingStore.addAll(embeddings, segments);
```


110

Présentation des types

- `DocumentParser` : sait lire les données d'un certain type ; par exemple les données au format PDF ou les données au format Office Microsoft
- « *`UnDocumentLoader`* » : crée un `Document` à partir de données provenant d'une source (fichier local, page Web, ...) au format supporté par le `DocumentParser`
- `DocumentSplitter` : découpe un `Document` en une liste de chunks (`List<TextSegment>`)
- `EmbeddingModel` : modèle d'embeddings qui peut convertir du texte en `Embedding` ; peut créer une `List<Embedding>` à partir d'une `List<TextSegment>`

111

- Création des documents

112

Interface `Document`

- Package dev.langchain4j.data.document
- Représente un texte ; souvent le contenu d'un fichier local ou d'une page Web
- Le format du fichier peut être un simple fichier texte, un PDF, un docx, ...
- Des métadonnées peuvent être attachées au document (classe `Metadata` qui est une enveloppe pour une `Map`) ; par exemple, la source du document, sa date de création, son auteur

113

Classe `TextSegment`

- Package dev.langchain4j.data.segment
- Représente un segment de texte d'une entité plus large, avec ses éventuelles métadonnées
- Création avec méthodes `static from` auquel on passe le texte d'origine :
  - `from(String texte, Metadata metadata)` (`metadata` est un paramètre optionnel)
- Getters
  - `String text()` pour extraire le contenu
  - `Metadata metadata()`

114

Classe Metadata

- Package dev.langchain4j.data.document
- Métadonnées d'un `Document` ou d'un `TextSegment`
- Pour un document, ça peut être la date de création, le propriétaire, …
- Pour un segment, ça peut être un numéro de page, la position du segment dans le document, le chapitre, …
- Les métadonnées sont enregistrées comme une `Map` avec les clés de type `String` et les valeurs de type `String`, `UUID`, `Long`, `Float`, `Double` (valeur `null` interdite)

115

Classe DocumentLoader

- Package dev.langchain4j.data.document
- Pour charger un document
- Une seule méthode `static` pour charger le document

```
Document load(DocumentSource source,
              DocumentParser parser)
```


116

Interface DocumentSource

- Package dev.langchain4j.data.document
- Source pour obtenir un `Document`
- 2 méthodes :
  - `InputStream inputStream() throws IOException` lit le contenu du document
  - `Metadata metadata()` retourne les métadonnées associées avec la source du document
- Nombreuses implémentations : `FileSystemSource`, `UrlSource`, `GitHubSource`, `AmazonS3Source`, …

117

Interface DocumentParser

- Package dev.langchain4j.data.document
- Pour parser un `InputStream` en un `Document`
- Une seule méthode `Document parse(InputStream inputStream)`
- Plusieurs implémentations : `ApachePdfBoxDocumentParser` (fichiers PDF), `ApachePoiDocumentParser` (fichiers doc, docx, ppt, pptx, xls, xlsx), `ApacheTikaDocumentParser` (fichiers PDF, doc, ppt, xls), `TextDocumentParser` (fichiers texte)

118

Classes utilitaires

- Des classes permettent de charger des documents des types les plus utilisés, sans implémenter `DocumentSource` :
  - `FileSystemDocumentLoader`
  - `ClassPathDocumentLoader`
  - `UrlDocumentLoader`
  - `GitHubDocumentLoader`
  - `AmazonS3DocumentLoader`
  - …

119

Exemples création document

```
// Charger un document local
Path path = Paths.get("/path/to/some/file.txt");
Document document =
    FileSystemDocumentLoader
    .loadDocument(path, new TextDocumentParser());

// Charger depuis le Web
URL url = new URL("https://...");
Document documentHtml =
    UrlDocumentLoader.load(url, new TextDocumentParser());
HtmlToTextDocumentTransformer transformer =
    new HtmlToTextDocumentTransformer(null, null, true);
Document documentText = transformer.transform(documentHtml);
```


120

FileSystemDocumentLoader

- Package dev.langchain4j.data.document.loader
- Classe utilitaire pour transformer en `Documents` des fichiers locaux dont on donne le chemin
- Méthodes `static` surchargées
  - `loadDocument` pour charger un document
  - `loadDocuments` pour charger les documents d'un répertoire
  - `loadDocumentsRecursively` pour charger les documents d'un répertoire (et des sous-répertoires)
- Chemin des fichiers ou du répertoire par `String` ou `Path`
- Paramètres optionnels pour le parser du document (`DocumentParser`) ou pour filtrer avec un pattern de nom de fichier (`PathMatcher`)

Exemples avec PathMatcher

```
• List<Document> documents = FileSystemDocumentLoader
      .loadDocuments("/home/langchain4j/documentation");
• PathMatcher pathMatcher = FileSystems.getDefault()
      .getPathMatcher("glob:*.pdf");
  List<Document> documents = FileSystemDocumentLoader
      .loadDocuments("/home/langchain4j/documentation",
                     pathMatcher);
• PathMatcher pathMatcher = FileSystems.getDefault()
      .getPathMatcher("glob:**.pdf");
  List<Document> documents = FileSystemDocumentLoader
      .loadDocumentsRecursively(
          "/home/langchain4j/documentation",
          pathMatcher);
```


Classe FileSystemSource

- Package dev.langchain4j.data.document.source
- Juste pour donner un exemple d'implémentation de `DocumentSource`
- Constructeur pour prend un `java.nio.file.Path` en paramètre ; on peut aussi utiliser les méthodes static from avec un de ces paramètres : `File`, `String`, `URI`, `Path`
- Méthodes :
  - `InputStream inputStream()`
  - `Metadata metadata()`

ClassPathDocumentLoader (1/2)

- Package dev.langchain4j.data.document.loader
- Crée un Document à partir du contenu de fichiers placés dans le classpath de l'application
- Méthodes `static` surchargées
  `loadDocument(String cheminFichier)` et
  `loadDocuments(String cheminRépertoire)`
  `loadDocumentsRecursively(String cheminRépertoire)`
  pour récupérer un `Document` à partir d'un fichier ou une `List<Document>` à partir de plusieurs fichiers dans un répertoire ; les chemins sont relatifs à un répertoire du classpath

ClassPathDocumentLoader (2/2)

- Ces méthodes surchargées peuvent prendre des paramètres supplémentaires pour
  - parser le fichier avec un DocumentParser ; par exemple, pour parser un fichier PDF
  - filtrer les fichiers avec un PathMatcher

Exemple

```
DocumentParser documentParser =
    new ApacheTikaDocumentParser();
Document document =
    ClassPathDocumentLoader.loadDocument("fichier.pdf",
                                         documentParser);
```

UrlDocumentLoader

- 2 méthodes `load static` surchargées :
  - `Document load(String url, DocumentParser)`
  - `Document load(java.net.URL url, DocumentParser)`
- Le `DocumentParser` à utiliser est le plus souvent un `TextDocumentParser`
- Si on veut ensuite ne pas récupérer les balises HTML, il faut transformer le `Document` avec un `HtmlToTextDocumentTransformer` (voir exemple déjà donné sur DocumentLoader)

HtmlToTextDocumentTransformer

- Implémente `DocumentTransformer` (une seule méthode `Document transform(Document)`
- Le constructeur peut prendre en paramètre
  - Un sélecteur CSS (type `String`), par exemple « #page-content » pour extraire le texte d'une partie de la page
  - Une `Map<String, String>` de sélecteurs CSS. Une clé peut être « titre » et sa valeur « #page-title » pour extraire le texte de l'élément d'id « #page-title » et l'enregistrer en métadonnée de clé « titre »
  - Un `boolean` qui indique si les liens de la page (pas les contenus des liens) doivent être inclus dans le texte extrait

- Découper les `Documents` en morceaux

Interface DocumentSplitter

- Package dev.langchain4j.data.document
- Découpe un document en segments de texte
- Indispensable car les LMs limitent la taille du contexte, et pour améliorer la précision des réponses
- Nombreuses classes d'implémentation parmi lesquelles `DocumentByParagraphSplitter`, `DocumentBySentenceSplitter`, `DocumentByWordSplitter`, `DocumentByRegexSplitter`, `HierarchicalDocumentSplitter`

Classe DocumentSplitters

- Package dev.langchain4j.data.document.splitter
- Classe utilitaire pour créer un `DocumentSplitter`
- Contient 2 méthodes `static recursive` qui retournent un `DocumentSplitter` pour découper un document en segments
- Ce splitter essaie de découper le document en paragraphes et met le plus de paragraphes possibles dans chaque `TextSegment`
- Si un paragraphe est trop long pour tenir dans un segment, il est récursivement découpé en lignes, puis en phrases, puis en mots et en caractères pour tenir dans un segment
- Paramètres des méthodes : taille maxi d'un segment (en caractères ou en tokens), taille maxi du chevauchement (seulement les phrases entières sont envisagées), tokenizer (optionnel) qui compte les tokens dans le texte

Interface Tokenizer

- Package dev.langchain4j.model
- Estime à l'avance le nombre de tokens d'un texte généré par un processus ; utilisé si la taille maximum d'un segment est donnée en tokens

Code avec splitter

```
URL fileUrl = TestRAG.class.getResource("/...");
Path path = Paths.get(fileUrl.toURI());
Document document = FileSystemDocumentLoader
    .loadDocument(path, new TextDocumentParser());

DocumentSplitter splitter =
    // découpe en segments de 600 caractères,
    // sans chevauchement
    DocumentSplitters.recursive(600, 0);

List<TextSegment> segments = splitter.split(document);
List<Embedding> embeddings =
    embeddingModel.embedAll(segments).content();

embeddingStore.addAll(embeddings, segments);
```


Métadonnées

- Les splitters de LangChain4j ajoutent automatiquement des métadonnées aux segments générés :
  - file_name et url : document qui a été découpé
  - index : position du segment dans le document
- Il est possible d'ajouter d'autres métadonnées à tous les segments générés en les ajoutant au document, avant le découpage en morceaux
- Il est aussi possible d'ajouter des métadonnées à un segment particulier, après le découpage

Exemple (1/2)

```
String path = "example.txt";

// Ajout métadonnées lors du chargement du document
Document document =
    FileSystemDocumentLoader.loadDocument(
        Paths.get(path),
        Metadata.from("un_nom_md", "une_valeur"));

// Splitter récursif (ajoute "index" automatiquement)
DocumentSplitter splitter =
    new RecursiveDocumentSplitter(300, 0);
List<TextSegment> segments = splitter.split(document);
```


Exemple (2/2)

```
// Affichage métadonnées de chaque segment
for (TextSegment segment : segments) {
  System.out.println("Segment :\n" + segment.text());
  System.out.println("Métadonnées :");
  segment.metadata().asMap().forEach(
      (key, value) ->
           System.out.println("  " + key + " = " + value));
  System.out.println("----------");
}
```


Utilisation des métadonnées

- 2 exemples sont donnés dans la documentation de LangChain4J
- **Filtrage par métadonnées :** https://github.com/langchain4j/langchain4j-examples/blob/main/rag-examples/src/main/java/_3_advanced/_05_Advanced_RAG_with_Metadata_Filtering_Examples.java
- A**jouter le nom du fichier dans les métadonnées** : https://github.com/langchain4j/langchain4j-examples/blob/main/rag-examples/src/main/java/_3_advanced/_04_Advanced_RAG_with_Metadata_Example.java

- Création des embeddings des segments et enregistrement dans un magasin d'embeddings

Interface EmbeddingModel

- Package dev.langchain4j.model.embedding
- Modèle d'embeddings qui crée des embeddings à partir de segments ; implémenté par de très nombreuses classes
- Méthode abstraite
  `Response<List<Embedding>> embedAll(List<TextSegment> textSegments)`
- Méthodes par défaut (`default`) :
  - `int dimension()` : renvoie dimension des embeddings
  - `Response<Embedding> embed(String text)` : renvoie embedding du texte
  - `Response<Embedding> embed(TextSegment segment)`

Exemples création EmbeddingModel

- ```
  EmbeddingModel embeddingModel =
      new AllMiniLmL6V2EmbeddingModel();
  ```
- ```
  EmbeddingModel embeddingModel =
      GoogleGenAiEmbeddingModel.builder()
        .modelName("gemini-embedding-001")
        .taskType(
          GoogleGenAiEmbeddingModel
              .TaskTypeEnum.RETRIEVAL_DOCUMENT)
        .apiKey(cle)
        .build();
  ```
- ```
  EmbeddingModel embeddingModel =
      OllamaEmbeddingModel.builder()
        .baseUrl("http://localhost:11434")
        .modelName("llama2")
        .build();
  ```


Obtenir un embedding

```
TextSegment segment = TextSegment.from("un texte …");
Response<Embedding> reponse =
    embeddingModel.embed(segment)
Embedding embedding = reponse.content();
```

Pourquoi `Response` ?
Pourquoi ne pas obtenir directement un embedding ?

Classe Response<T>

- Package dev.langchain4j.model.output
- Représente une réponse de plusieurs types de modèle (embedding, image, modération, scoring)
- Méthodes :
  - `@NonNull T content()` : récupère le contenu
  - `TokenUsage tokenUsage()` : récupère les statistiques d'usage
  - `FinishReason finishReason()`
  - `@NonNull Map<String,Object> metadata()` : récupère les éventuelles métadonnées

EmbeddingStore<T> (1/2)

- Interface du package dev.langchain4j.store.embedding
- Pour un magasin/dépôt d'embeddings
- `T` représente la classe de ce qui va être transformé en embeddings, typiquement `TextSegment`
- Méthodes `add` et `addAll` pour ajouter un ou plusieurs embeddings, avec ou sans leur texte d'origine
- Méthodes `remove` et `removeAll` pour supprimer des embeddings
- Implémenté par des classes adaptées aux BD vectorielles (Neo4j, Chroma, MongoDB,…) et par la classe `InMemoryEmbeddingStore`

EmbeddingStore<T> (2/2)

- Méthode pour chercher des embeddings similaires (pour la phase « retrieval » du RAG) ; ce que l'on cherche est défini par `EmbeddingSearchRequest` :
  ```
  default EmbeddingSearchResult<Embedded>
          search(EmbeddingSearchRequest request)
  ```


  ```

InMemoryEmbeddingStore<T>

- Package dev.langchain4j.store.embedding.inmemory
- Entrepôt pour embeddings (représentés par T), en mémoire centrale ; pour tests rapides sans BD vectorielle
- Pas d'indexation des embeddings ; ils sont parcourus du premier au dernier pour la recherche des embeddings similaires

Création EmbeddingStore

```
EmbeddingStore<TextSegment> embeddingStore =
    new InMemoryEmbeddingStore();
// ou
EmbeddingStore<TextSegment> embeddingStore =
    ChromaEmbeddingStore.builder()
        .baseUrl("http://localhost:8000")
        .collectionName("my-collection")
        .build();
```


Exemple ajout embeddings

```
EmbeddingModel embeddingModel =
    new AllMiniLmL6V2EmbeddingModel();
Response<List<Embedding>> response =
    embeddingModel.embedAll(segments);
List<Embedding> listeEmbedding = response.content();

EmbeddingStore<TextSegment> embeddingStore =
    new InMemoryEmbeddingStore<>();
embeddingStore.addAll(listeEmbedding, segments);
```


Sérialisation BD vectorielle en mémoire

- Pendant les tests de code pour le RAG, il est possible de n'effectuer la phase d'ingestion qu'une seule fois dans la BD vectorielle en mémoire, pour éviter de dépenser des tokens pour la création des embeddings
- Il est possible de sauvegarder le contenu de la base dans un fichier JSON et de récupérer ce contenu lors des tests suivants, si le fichier existe

Méthodes pour la sérialisation

- `void serializeToFile(String filePath)`
- `void serializeToFile(Path filePath)`
- `String serializeToJson()`
- `static InMemoryEmbeddingStore<TextSegment> fromFile(String filePath)`
- `static InMemoryEmbeddingStore<TextSegment> fromFile(Path filePath)`
- `static InMemoryEmbeddingStore<TextSegment> fromJson(String json)`

Exemple de code

```
Path path = Path.of("data/embedding-store.json");
InMemoryEmbeddingStore<TextSegment> embeddingStore;

if (Files.exists(path)) {
  // On recharge les embeddings déjà calculés
  embeddingStore = InMemoryEmbeddingStore.fromFile(path);
}
else {
  // Code ingestion
  ...
  // Sauvegarde embeddingStore pour les prochaines fois
  embeddingStore.serializeToFile(path);
}
```

Encapsuler dans méthode (1/2)

```
static InMemoryEmbeddingStore<TextSegment>
chargerOuCreer(Path path,
               List<Document> documents,
               EmbeddingModel embeddingModel)
throws IOException {
  if (Files.exists(path)) {
    System.out.println("Chargement embeddings...");
    return InMemoryEmbeddingStore.fromFile(path);
  }
```


151

Encapsuler dans méthode (2/2)

```
  // Code pour ingestion si 1ère fois
  InMemoryEmbeddingStore<TextSegment> store =
      new InMemoryEmbeddingStore<>();
  EmbeddingStoreIngestor ingestor = ...;
  ingestor.ingest(documents);

  // Crée répertoire s'il n'existe pas déjà
  if (path.getParent() != null) {
    Files.createDirectories(path.getParent());
  }
  store.serializeToFile(path);

  return store;
}
```


152

Classe `EmbeddingSearchRequest`

- Représente une requête dans un `EmbeddingStore`
- Builder et constructeur avec les paramètres de type
  - `Embedding` : embedding dont on cherche les embeddings similaires
  - `Integer` : nombre maximum d'embeddings à retourner (3 par défaut)
  - `Double` : seulement les embeddings avec un score >= à ce nombre seront retournés (compris entre 0 et 1, bornes comprises ; 0 par défaut)
  - `Filter` : filtre à appliquer aux **métadonnées** ; seuls les segments qui correspondent à ce filtre seront retournés ; pas de filtre par défaut

153

`EmbeddingSearchResult<T>`

- Package dev.langchain4j.store.embedding
- Classe générique (paramètre de type nommé `T`, `TextSegment` le plus souvent) qui représente le résultat d'une recherche dans un magasin d'embeddings
- Constructeur qui prend en paramètre une `List<EmbeddingMatch<T>>`
- Instance retournée par la méthode `search` de `EmbeddingStore<T>`
- Méthode `matches()` qui retourne une `List<EmbeddingMatch<T>>`

154

`EmbeddingMatch<T>`

- Classe qui représente un embedding retourné par une recherche, avec son score de pertinence (dérivé de la distance cosinusoïdale) par rapport à l'embedding de la recherche, son ID et son contenu d'origine (`TextSegment` le plus souvent)
- Méthodes :
  - `T embedded()` retourne le contenu d'origine
  - `Embedding embedding()` retourne l'embedding
  - `String embeddingId()` retourne id embedding dans magasin
  - `Double score()` retourne le score de pertinence

155

Exemple recherche plus proche

```
Embedding embeddingQuestion = embeddingModel.embed("Quel est
votre sport favori ?").content();
EmbeddingSearchRequest searchRequest =
    EmbeddingSearchRequest.builder()
        .queryEmbedding(embeddingQuestion)
        .maxResults(3)
        .build();
EmbeddingSearchResult<TextSegment> resultatRecherche =
    embeddingStore.search(searchRequest);
List<EmbeddingMatch<TextSegment>> pertinents =
    resultatRecherche.matches();
pertinents.forEach(pertinent -> {
    System.out.println(pertinent.score());
    System.out.println(pertinent.embedded().text());
});
```


156

EmbeddingStoreIngestor (1/3)

- Package dev.langchain4j.store.embedding
- Pipeline responsable de l'ingestion des documents dans un magasin d'embeddings
- Il gère tout le processus : découpage en segments, génération des embeddings pour chaque segment, enregistrement des embeddings
- Il est possible d'ajouter à ce processus la transformation des documents et la transformation des segments après le découpage en segments
- Les informations nécessaires aux traitements sont indiquées à la création de l'ingestor

157

EmbeddingStoreIngestor (2/3)

- La création de l'ingestor se fait par un builder ou un constructeur ; on indique le modèle et le magasin pour les embeddings, le splitter, des éventuels transformeurs pour le document ou/et les segments :
  - `DocumentTransformer` peut transformer le document (nettoyage, filtrage, enrichissement, extraire le texte du contenu HTML, métadonnées, …) ; optionnel
  - `DocumentSplitter` ; optionnel
  - `TestSegmentTransformer` peut transformer les segments, par exemple pour ajouter des métadonnées
  - `EmbeddingModel`
  - `EmbeddingStore`

158

EmbeddingStoreIngestor (3/3)

- L'ingestion se lance avec une des méthodes `ingest`
- La méthode `ingest` prendre en paramètre un `Document` ou une liste de `Documents`
- 2 méthodes `ingest` sont `static` ; permettent d'indiquer un `EmbeddingStore<TextSegment>` ; utilisent un `EmbeddingModel` et un `DocumentSplitter` par défaut
- La valeur retour des méthodes `ingest` est de type `IngestionResult`, ce qui permet de récupérer un `TokenUsage` (qui donne le nombre de tokens utilisés pendant la création des embeddings)

159

Exemple ingestion avec EmbeddingStoreIngestor

```
DocumentSplitter splitter =
    DocumentSplitters.recursive(600, 0);
EmbeddingModel embeddingModel =
    new AllMiniLmL6V2EmbeddingModel();
EmbeddingStore<TextSegment> embeddingStore =
    new InMemoryEmbeddingStore<>();
EmbeddingStoreIngestor ingestor =
    EmbeddingStoreIngestor.builder()
        .documentSplitter(splitter)
        .embeddingModel(embeddingModel)
        .embeddingStore(embeddingStore)
        .build();
Document document = FileSystemDocumentLoader
    .loadDocument(path, new TextDocumentParser());
ingestor.ingest(document);
```

Explications ?

160

Exemple avec transformations

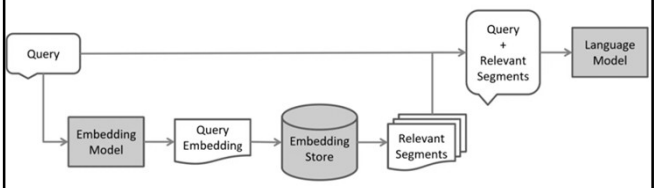
```
EmbeddingStoreIngestor ingestor =
    EmbeddingStoreIngestor.builder()
    .embeddingModel(embeddingModel)
    .embeddingStore(embeddingStore)
    .documentSplitter(documentSplitter)
    .documentTransformer(document -> {
        document.metadata().put("user", …);
        return document;
     })
    .textSegmentTransformer(segment -> TextSegment.from(
        segment.metadata().getString(file_name) + "\n"
         + segment.text(), segment.metadata()))
    .build();

List<Document> docs = FileSystemDocumentLoader
    .loadDocumentsRecursively(path);
ingestor.ingest(documents);
```

Explications ?

161

RAG - récupération et augmentation



Source https://docs.langchain4j.dev/tutorials/rag

Explications ?

162

Exemple schématique simple

```
ContentRetriever contentRetriever =
     EmbeddingStoreContentRetriever.builder()
         .embeddingStore(embeddingStore)
         .embeddingModel(embeddingModel)
         .maxResults(10) // Nombre de résultats à retourner
         .minScore(0.75) // Score minimal pour similarité
         .build();
ChatMemory chatMemory = ... ;
Assistant assistant = AiServices.builder(Assistant.class)
    .chatModel(model)
    .chatMemory(chatMemory)
    .contentRetriever(contentRetriever)
    .build();
System.out.println(assistant.chat("..."));
```


Récupération

- Dans les cas simples comme dans l'exemple précédent l'assistant IA est configuré avec la méthode du builder `contentRetriever`
- Pour les cas plus complexes, configurer avec la méthode `retrievalAugmentor`
- Il faut choisir ; une erreur sera lancée si l'assistant est configuré avec les 2 méthodes

Interface `ContentRetriever`

- Retrouve des contenu (`Content`) d'une source de données, pertinents pour un `Query`
- La source de données peut être
  - un magasin d'embeddings
  - un moteur de recherche dans du texte ou sur le Web
  - une API
  - une BD relationnelle
  - etc.
- Une seule méthode qui retourne les contenus retrouvés, triés par pertinence, les plus pertinents en premiers `List<Content> retrieve(Query)`

Des implémentations de `ContentRetriever`

- `EmbeddingStoreContentRetriever` : retrouve depuis un `EmbeddingStore`
- `WebSearchContentRetriever` : retrouve depuis le Web, en utilisant un `WebSearchEngine`
- `SqlDatabaseContentRetriever` : retrouve dans une BD relationnelle ; génère des requêtes SQL à partir du langage naturel (attention, danger !)
- `Neo4jContentRetriever` : utilise Neo4j, une BD orientée graphe
- `AzureAiSearchContentRetriever` : utilise le service de recherche de Azure (fournisseur de cloud)



Classe `Query`

- Package dev.langchain4j.rag.query
- Représente une question posée par l'utilisateur pour retrouver des `Contents`
- 2 constructeurs avec ces paramètres (on peut aussi utiliser les méthodes `static from` avec les mêmes paramètres) :
  - `String`
  - `String` et `Metadata`
- 2 méthodes
  - `String txt()`
  - `Metadata metadata()`

Classe `Metadata`

- Package dev.langchain4j.rag.query ; ne pas confondre avec la classe du package dev.langchain4j.data.document
- Métadonnées utilisées pour retrouver des informations (utilisée par `Query`) ou pour augmenter une question posée par l'utilisateur (avant d'envoyer la requête au LM)
- Constructeur (ou méthode `static from`) avec les paramètres de type `UserMessage`, `Object` (`chatMemoryId` peut être utilisé pour distinguer les utilisateurs), `List<ChatMessage>` (chatMemory pour les messages précédents du chat, pour fournir du contexte)
- 3 « getters » `userMessage()`, `chatMemoryId()`, `chatMemory()`

Content

- Package dev.langchain4j.rag.content
- Représente un contenu pertinent pour un `Query`
- Uniquement pour `TextSegment` pour le moment, en attendant d'autres types de données (images, audio, …)
- Constructeur avec un paramètre de type `String` ou `TextContent` ; une méthode `static from` avec les mêmes paramètres permet aussi de créer une instance
- Méthode `TextSegment textSegment()` pour récupérer le `TextSegment`

EmbeddingStoreContentRetriever

- Classe du package dev.langchain4j.rag.content.retriever
- `EmbeddingStoreContentRetriever` : retrouve depuis un `EmbeddingStore` ; par défaut, retrouve les 3 `Contents` les plus pertinents, sans filtre ; créé avec un builder
- Méthodes du builder pour donner un nom, un nombre maximum de résultats, un score minimum de pertinence, un filtre (`Filter`) pour les métadonnées, un filtre dynamique (fonction qui peut dépendre du `Query`, de l'utilisateur, …)

Interface Filter

- Package dev.langchain4j.store.embedding.filter
- Pour filtrer les **métadonnées**
- Seule la méthode `boolean test(Object)` est abstraite et on peut donc utiliser une expression lambda ; elle teste si un objet satisfait le filtre
- Méthodes `static and`, `not`, `or` ; méthode default `and`, `or` qui correspondent aux méthodes `static`, avec `this` comme filtre gauche
- Implémentée par `And`, `IsEqualTo`, `IsGreaterThan`, `IsGreaterThanOrEqualTo`, `IsIn`, `IsLessThan`, `IsLessThanOrEqualTo`, `IsNotEqualTo`, `IsNotIn`, `IsTextMatch`, `IsTextMatchPhrase`, `Not`, `Or`

Exemple de filtre dynamique

```
Function<Query, Filter> filterByUserId =
   (query) -> metadataKey("userId")
              .isEqualTo(query.metadata()
              .chatMemoryId().toString());
...
ContentRetriever contentRetriever =
    EmbeddingStoreContentRetriever.builder()
    .embeddingStore(embeddingStore)
    .embeddingModel(embeddingModel)
    // La recherche sera limitée aux segments qui
    // appartiennent à l'utilisateur en cours
    .dynamicFilter(filterByUserId)
    .build();
```


WebSearchContentRetriever

- Classe du package dev.langchain4j.rag.content.retriever
- Retrouve depuis le Web, en utilisant un `WebSearchEngine`
- Méthodes du builder pour le nombre maximum de résultats et pour le `WebSearchEngine`
- La seule méthode est celle de l'interface `ContentRetriever` :
  `List<Content> retrieve(Query query)`

Interface WebSearchEngine

- Package dev.langchain4j.web.search
- Représente un moteur de recherche sur le Web
- 2 méthodes
  - `WebSearchResults search(WebSearchRequest webSearchRequest)`, la seule méthode abstraite
  - `default WebSearchResults search(String query)`
- Implémentée par les classes `GoogleCustomWebSearchEngine`, `SearchApiWebSearchEngine`, `TavilyWebSearchEngine`

Interface `WebSearchRequest`

- Package dev.langchain4j.web.search
- Représente une requête envoyée à un moteur de recherche sur le Web ; suit le standard OpenSearch
- Méthode principale pour créer une requête :
  - `from(String searchTerms[, Integer maxResults])`

175

Interface `WebSearchResults`

- Package dev.langchain4j.web.search
- Représente le résultat d'une requête de recherche sur le Web

176

Ce que peut faire un RAG avancé

- Modifier la question utilisée pour la recherche d'informations
- Choisir parmi plusieurs magasins d'embedding (pendant la phase 1 du RAG les données ont été enregistrées dans ces magasins)
- Faire du reranking : réordonner / sélectionner les informations récupérées en les examinant plus finement
- Utilisation d'un filtrage et classement différent, suivant le type de source utilisé
- …

177

Composants phase 2 du RAG

- RetrievalAugmentor : recherche les informations pertinentes et les ajoute à la question ; utilise en particulier un ContentRetriever
- Générateur : LM qui génère la réponse en utilisant ses capacités linguistiques et les informations pertinentes retrouvées

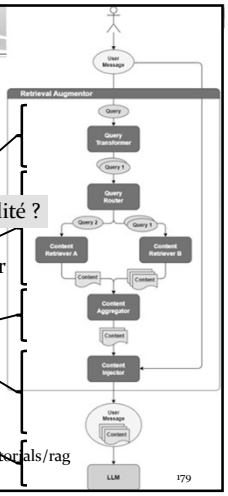
178

Explications ?

`RetrievalAugmentor`

- Schéma des différentes étapes :
  - Réécrire le prompt, par exemple en ajoutant un résumé de la conversation
  - Router vers les bons récupérateurs pour trouver les informations pertinentes
  - Agréger les informations retrouvées
  - Ajouter les informations au prompt
  - Passer le prompt augmenté au LM

Utilité ?



Source image https://docs.langchain4j.dev/tutorials/rag

179

Utilisation `RetrievalAugmentor`

- Le désigner lors de la création d'un assistant IA :

```
Assistant assistant =
  AiServices.builder(Assistant.class)
  ...
  .retrievalAugmentor(retrievalAugmentor)
  .build();
```


180

Interface `RetrievalAugmentor`

- Package dev.langchain4j.rag
- Une seule méthode :
  - `AugmentationResult augment(AugmentationRequest augmentationRequest)`
  augmentationRequest contient le `ChatMessage` à augmenter
- Implémentée par `DefaultRetrievalAugmentor` qui devrait convenir dans la majorité des cas



181

Class `AugmentationRequest`

- Package dev.langchain4j.rag
- Représente une requête pour une augmentation de `ChatMessage`
- Constructeur avec paramètres de type `ChatMessage` et `Metadata` (du package `dev.langchain4j.rag.query`)
- 2 « getters » `ChatMessage chatMessage()`, `MetaData metadata()`

182

Class `AugmentationResult`

- Package dev.langchain4j.rag
- Représente le résultat d'une augmentation
- 2 getters :
  - `ChatMessage chatMessage()` : message d'origine, éventuellement récrit
  - `List<Content> contents()` : les ajouts

183



`DefaultRetrievalAugmentor` (1/2)

- Package dev.langchain4j.rag
- Classe qui organise le flot entre ces composants :
  1. Un `QueryTransformer` pour transformer le `Query` (obtenu avec le `UserMessage`, le prompt de départ)
  2. Un `QueryRouter` va router le `Query` vers un ou plusieurs des `ContentRetriever`s
  3. Les `Content` retrouvés sont alors transformés par un `ContentAggregator` en une liste finale de `Content`
  4. Un `ContentInjector` ajoute la liste au `UserMessage`

184

`DefaultRetrievalAugmentor` (2/2)

- Chaque composant (à part `ContentRetriever`) est initialisé avec une classe d'implémentation par défaut ; par exemple `DefaultQueryTransformer` ou `DefaultQueryRouter`
- D'autres classes avancées d'implémentation sont fournies par LangChain4j, par exemple `CompressingQueryTransformer` ou `ExpandingQueryTransformer`

185

`QueryTransformer`

- Interface du package dev.langchain4j.rag.query.transformer
- Permet de réécrire une question : transforme un `Query` en un ou plusieurs `Query` (utilise souvent une IA)
- Implémentée par `DefaultQueryTransformer` (pas de transformation), `CompressingQueryTransformer` et `ExpandingQueryTransformer`
- Pour plus de détails : https://blog.langchain.dev/query-transformations/

186

Pourquoi réécrire le prompt ?

- Une des raisons est que le RAG simple récupère les morceaux similaires à la **dernière** question
- Si cette dernière question a besoin du contexte de la conversation pour être bien comprise, les segments récupérés risquent de ne pas être utiles

Exemple

- Q1 : Est-ce qu'un client peut résilier sans frais ?
- Réponse : Oui.
- Q2 : Et quelle est la procédure ?
- Pour la 2ème question, le RAG va ramener les segments similaires à Q2 (sans référence à la résiliation sans frais), qui seront vraisemblablement en grande partie inutiles pour améliorer la réponse du LM
- De plus ce problème peut passer inaperçu car le LM utilisera l'historique de la conversation pour donner une réponse à peu près correcte (mais moins précise que si le RAG avait ajouté les bonnes informations à la question)

`CompressingQueryTransformer`

- Fait modifier le `Query` (la dernière question) par un `ChatModel`, pour le compresser avec l'historique de la conversation (ajoute donc l'envoi d'une requête à ce modèle, en plus de la requête initiale)
- Utilise un `ChatModel` pour compresser le query avec l'historique enregistré dans la mémoire
- Constructeur (ou builder) qui prend en paramètres un `ChatModel`, et optionnellement un `PromptTemplate` pour indiquer au modèle comment compresser

Exemple

- Avec l'exemple précédent, la dernière question sera reformulée « Quelle est la procédure pour qu'un client résilie sans frais ? » avant d'aller chercher les morceaux similaires dans la base de données vectorielle

Exemple de code

```
QueryTransformer queryTransformer =
    CompressingQueryTransformer.builder()
        .chatModel(modele)
        .build();
...
RetrievalAugmentor retrievalAugmentor =
    DefaultRetrievalAugmentor.builder()
        .queryTransformer(queryTransformer)
        .contentRetriever(retriever)
        .build();
Assistant assistant = AiServices.builder(Assistant.class)
        .chatModel(modele)
        .retrievalAugmentor(retrievalAugmentor)
        . ...;
```



Avertissement

- Comme la compression ajoute un appel à un LM, il ne faut pas systématiquement l'utiliser, même si un petit modèle local est souvent suffisant
- Plusieurs pratiques permettent de n'ajouter la compression que lorsqu'il le faut : heuristiques pour savoir si la question ne se suffit pas à elle-même (courte, utilisation de pronoms ou de mots comme « ça », « et pour », « y », ...), évidemment inutile s'il n'y a qu'une question, ...
- Si on l'utilise au cas par cas pour chaque question, il faut écrire un `QueryTransformer` personnalisé

ExpandingQueryTransformer

- Utilise un `ChatModel` pour étendre le query en plusieurs query
- Constructeur qui prend en paramètres un `ChatModel`, et optionnellement
  - un `PromptTemplate` pour indiquer au modèle comment étendre
  - le nombre de `Query` (3 par défaut)

QueryRouter

- Interface du package dev.langchain4j.rag.query.router
- Une seule méthode :
  `Collection<ContentRetriever> route(Query query)`
  qui renvoie un ou plusieurs `ContentRetriever`, selon la question (`Query`)
- Le choix du ou des `ContentRetriever` peut être fait par un LM, par un modèle d'embeddings, par des mots-clés, selon l'utilisateur qui pose la question (`query.metadata().chatMemoryId()`), par les autorisations de l'utilisateur, ...
- Implémentée par 2 classes `DefaultQueryRouter` `LanguageModelQueryRouter`

DefaultQueryRouter

- Package dev.langchain4j.rag.query.router
- Constructeur qui prend en paramètre un ou plusieurs `ContentRetriever`
- La méthode `route` renvoie tous les `ContentRetriever` ; on peut la redéfinir dans des classes filles en utilisant le texte ou les métadonnées de `Query`

LanguageModelQueryRouter

- Choix des `ContentRetriever`s fait par un LM
- Si la connexion au LM ou si le LM donne une réponse non valable, on peut donner une stratégie de fallback (de repli) : pas de RAG, lancer une exception, router vers tous les `ContentRetriever`s
- Builder pour créer une instance ; on peut passer
  - une `Map<ContentRetriever, String>` qui décrit chaque `ContentRetriever`
  - un `ChatLanguageModel`, celui qui décide
  - un `PromptTemplate` pour poser la question au LM
  - une stratégie de repli

Exemple de routage

```
// Création de 2 ContentRetrievers pour retrouver
// les informations à ajouter au prompt
...
// Descriptions associées à chacun des ContentRetriever
Map<ContentRetriever, String> descriptions =
    new HashMap<>();
descriptions.put(contentRetriever1,
                 "Conditions annulation contrat");
descriptions.put(contentRetriever2,
                 "Règles pour travail à distance");
QueryRouter queryRouter =
    new LanguageModelQueryRouter(chatModel,
                                 descriptions);
```

Les descriptions aideront le LM à choisir le bon `ContentRetriever` pour chaque question posée

Exemple utilisation QueryRouter

```
QueryRouter queryRouter = ...;

RetrievalAugmentor retrievalAugmentor =
    DefaultRetrievalAugmentor.builder()
                .queryRouter(queryRouter)
                .build();

Assistant assistant = AiServices.builder(Assistant.class)
                .chatModel(model)
                .retrievalAugmentor(retrievalAugmentor)
                .build();
```

QueryRouter personnalisé

- Il faut implémenter l'interface `QueryRouter`
- Donc implémenter la méthode `Collection<ContentRetriever> route(Query query)`
- Le paramètre est la question envoyée au LM ; la méthode retourne le ou les `ContentRetriever` qui seront utilisés pendant la phase de récupération du RAG
- Souvent la classe d'implémentation est une classe interne locale à la méthode qui l'utilise ; classe anonyme ou pas
- Dans l'exemple suivant le choix est fait par un LM : pas de RAG si la question ne porte pas sur l'IA

Exemple classe interne locale

```
class QueryRouterPourEviterRag implements QueryRouter {
  @Override
  public List<ContentRetriever> route(Query query) {
    String question =
        "Est-ce que la requête '" + query.text()
        + "' porte sur l'IA ? Réponds seulement par"
        + " 'oui', 'non', ou 'peut-être'.";
    String reponse = model.chat(question);
    if (reponse.toLowerCase().contains("non")) {
      // Pas de RAG (aucun ContentRetriever utilisé)
      return Collections.emptyList();
    } else {
      return Collections.singletonList(contentRetriever);
    }
  }
}
```



Exemple modification template prompt

```
RetrievalAugmentor retrievalAugmentor =
      DefaultRetrievalAugmentor.builder()
          .contentRetriever(contentRetriever)
          .contentInjector(
              DefaultContentInjector.builder()
                  .promptTemplate(
                      PromptTemplate.from("""
   {{userMessage}}

   Tu dois répondre uniquement à partir des infos ci-dessous
   Si elles sont insuffisantes, réponds : 'JE NE SAIS PAS'.

   === CONTEXTE ===
   {{contents}}
   """))
                  .build())
          .build();
```


Interface ContentAgregator

- Package dev.langchain4j.rag.content.aggregator
- Réunit les `Contents` retrouvés par tous les `ContentRetriever` afin de sélectionner les plus pertinents
- Par exemple pour faire du reranking
- Une méthode `List<Content> aggregate(Map<Query,Collection<List<Content>>> queryToContents)`
- Implémentée par `DefaultContentAggregator`, `ReRankingContentAggregator`

DefaultContentAgregator

- Implémentation de `ContentAgregator` qui convient dans la plupart des cas

ReRankingContentAggregator

- Package dev.langchain4j.rag.content.aggregator
- Effectue un reranking en utilisant un `ScoringModel` (modèle spécialisé dans le reranking)
- Peut-être configuré en donnant le score minimum d'un `Content` pour qu'il soit retenu

Interface `ScoringModel`

- Package dev.langchain4j.model.scoring
- Attribue un score de similarité par rapport à un texte sous la forme d'une `Response`
- Une méthode abstraite
  `Response<List<Double>> scoreAll(List<TextSegment> segments, String query)`
- 2 méthodes `default`
  - `Response<Double> score(String text, String query)`
  - `Response<Double> score(TextSegment segment, String query)`

205

Exemple pour reranking (1/2)

```
ScoringModel scoringModel = CohereScoringModel.builder()
                .apiKey(cohereKey)
                .modelName("rerank-multilingual-v3.0")
                .build();

ContentAggregator contentAggregator =
    ReRankingContentAggregator.builder()
                .scoringModel(scoringModel)
                .minScore(0.6)
                .build();
```


206

Exemple pour reranking (2/2)

```
RetrievalAugmentor retrievalAugmentor =
    DefaultRetrievalAugmentor.builder()
                .contentRetriever(contentRetriever)
                .contentAggregator(contentAggregator)
                .build();

Assistant assistant =
    AiServices.builder(Assistant.class)
                .chatModel(modele)
                .retrievalAugmentor(retrievalAugmentor)
                .chatMemory(memory)
                .build();
```


207

Interface `ContentInjector`

- Package dev.langchain4j.rag.content.injector
- Injecte des `Contents` dans un `UserMessage`
- Permet d'ajouter au prompt les informations pertinentes récupérées dans les étapes précédentes
- Une seule méthode
  `ChatMessage inject(List<Content> contents, ChatMessage chatMessage)`

208

Classe `DefaultContentInjector`

- Package dev.langchain4j.rag.content.injector
- Implémentation par défaut de `DefaultContentInjector`
- Ajoute les `Contents` à la fin du `UserMessage`, dans l'ordre de la liste (voir `DEFAULT_PROMPT_TEMPLATE`)
- Constructeurs et pattern builder pour créer une instance ; on peut indiquer un template (avec variables `{{userMessage}}` et `{{contents}}`) et une liste de clés pour métadonnées qui seront ajoutées à chaque segment (`Content.textSegment()`)

209

Références

210

- RAG explained : embedding, sentence BERT, vector database par Umar Jamil : https://www.youtube.com/watch?v=rhZgXNdhWDY
- RAG et fine-tuning : https://learn.microsoft.com/fr-fr/azure/developer/ai/augment-llm-rag-fine-tuning
- RAG avancé : https://learn.microsoft.com/fr-fr/azure/developer/ai/advanced-retrieval-augmented-generation
- Quelques exemples d'utilisation du RAG : https://www.cohesity.com/fr/glossary/retrieval-augmented-generation-rag/

211

- Méthodes avancées de RAG : https://fr.linkedin.com/pulse/m%C3%A9thodes-avanc%C3%A9es-de-g%C3%A9n%C3%A9ration-augment%C3%A9e-par-rag-ilan-bompuis-ynoof
- Guide de fine-tuning de unsloth : https://unsloth.ai/docs/get-started/fine-tuning-llms-guide
- Stratégies de chunking : https://blog.bytebytego.com/i/220736007/8-chunking-strategies-for-rag-based-agents

212

Exemples LangChain4j

- https://github.com/langchain4j/langchain4j-examples/tree/main/rag-examples
- Méthodes avancées de RAG : https://github.com/langchain4j/langchain4j-examples/tree/main/rag-examples/src/main/java/_3_advanced

213